INTERNSHIP TRAINING REPORT

ON

INTERNSHIP ON CLOUD COMPUTING

(COMPUTER SCIENCE)

ORGANISED BY

DREAMZ VISION FINISHING

TRAINER'S SIGNATURE

SUBMITTED BY

Samriddha Das

NAME	:	**Samriddha Das**
REG. NO.	:	**D242520118**
SEM	:	**5TH**
YEAR	:	**2026-2027**
INSTITUTE NAME	:	**Kalna Polytechnic**
DEPARTMENT	:	**Computer Science and Technology**
FIELD OF TRAINING	:	**INTERNSHIP ON CLOUD COMPUTING (COMPUTER SCIENCE)**
PERIOD	:	**FROM 28/10/26 TO 06/11/26**
PROJECT NAME	:	**CloudLens - AWS S3 & EC2 Image Gallery Web App**
LIVE WEB URL	:	**http://32.236.11.79**
GITHUB LINK	:	**https://github.com/samriddha-coder/-CloudLens-AWS-hyphen-gallery-**

INTRODUCTION

Cloud Computing is the on-demand delivery of IT resources over the Internet with pay-as-you-go pricing. Instead of buying, owning, and maintaining physical servers and data centers, organizations can access technology services, such as compute power, storage, and databases, on an as-needed basis from a cloud provider like Amazon Web Services (AWS).

In modern Computer Science and Software Engineering, cloud infrastructure is foundational for deploying resilient, high-availability web applications. By utilizing cloud-native architectures, developers decouple frontend clients, backend APIs, relational databases, and static media storage into scalable microservices.

Amazon Web Services (AWS) is the world's most comprehensive and broadly adopted cloud platform. During this internship training, two primary AWS core services were integrated and deployed to build the **CloudLens** project:

- **Amazon Simple Storage Service (S3):** An industry-leading object storage service offering 99.999999999% (11 9s) of data durability. S3 is utilized to store, manage, and deliver user-uploaded images globally with fine-grained bucket policies and Cross-Origin Resource Sharing (CORS).
- **Amazon Elastic Compute Cloud (EC2):** Provides secure, resizable compute capacity in the cloud. An Ubuntu Linux virtual machine instance was provisioned to execute the containerized Node.js backend, React frontend, and PostgreSQL database cluster.
- **Containerization with Docker:** Encapsulates application code and its dependencies into lightweight containers, guaranteeing seamless parity between development and production cloud environments.

AIM

To design, develop, and deploy a full-stack, cloud-native web application named **CloudLens** using modern web technologies (React.js, Node.js, Express, PostgreSQL) and enterprise cloud infrastructure (AWS S3, AWS EC2, and Docker), implementing public image ingestion and viewing alongside Role-Based Access Control (RBAC) for administrative asset deletion.

CORE INTERNSHIP OBJECTIVES

1. **Cloud Infrastructure Provisioning:** Gain hands-on practical competence in creating, configuring, and securing Amazon S3 buckets and Amazon EC2 virtual machines in the ap-southeast-2 (Sydney) region.
2. **Direct S3 Media Streaming:** Implement server-side binary buffer streaming using the official AWS SDK (@aws-sdk/client-s3) to upload and delete objects without storing temporary files on disk.
3. **Relational Database Modeling:** Design a normalized PostgreSQL schema to persist image metadata (IDs, S3 URLs, original names, uploader identities, and timestamps).
4. **Role-Based Access Control (RBAC):** Implement JWT-based authentication ensuring that public users can only upload and view images, while destructive deletion operations are restricted strictly to administrators.
5. **DevOps Container Orchestration:** Architect a multi-container Docker Compose pipeline running PostgreSQL, Express API, and Nginx reverse proxy on an AWS EC2 Ubuntu instance.
6. **Version Control & Public Verification:** Version control all codebase artifacts on GitHub and verify live accessibility on public IPv4 **32.236.11.79**.

SYSTEM ARCHITECTURE OVERVIEW

Tier	Technology	Key Role in CloudLens
Client Layer	React.js (Vite), CSS3	Responsive web UI, drag-and-drop file upload, lightbox preview, and admin modal.
Application Layer	Node.js (Express REST API)	API routing, Multer memory buffer streaming, JWT authorization.
Cloud Storage	Amazon S3 (ap-southeast-2)	Durable object storage bucket (samriddhabaket1718) with CORS & public read.
Database Tier	PostgreSQL 16 Engine	Relational schema tracking asset metadata with persistent host volume mounting.
Compute / Host	AWS EC2 (Ubuntu 24.04)	Virtual cloud server hosting the entire Docker cluster at http://32.236.11.79.

FAMILIARIZATION WITH AWS CLOUD SERVICES

A foundational component of the internship was mastering the AWS Management Console navigation, Identity and Access Management (IAM), and security isolation principles.

Key Components Mastered:

1. **AWS Management Console:** Unified web-based portal to access, provision, and monitor Amazon Web Services across global geographic regions.
2. **Region Selection:** Configured compute and storage resources in the Asia Pacific (Sydney) ap-southeast-2 region for optimized latency.
3. **IAM Identity & Access Management:** Created dedicated programmatic service user gallery-app-user with least-privilege security principle.
4. **Access Key & Secret Key Generation:** Provisioned secure API credentials (AKIA...) enabling server-to-cloud API communications.
5. **Security Groups (Stateful Virtual Firewalls):** Configured inbound ingress rules controlling network protocols, ports, and allowed IP CIDR ranges.
6. **EC2 Instance Connect:** Established browser-based SSH terminal sessions directly to the running Linux instance.

AWS EC2 Inbound Network Security Configuration:

Type / Protocol	Port	Source	Operational Purpose
HTTP (TCP)	80	0.0.0.0/0	Standard Production Web Traffic (Nginx Reverse Proxy)
Custom TCP	3000	0.0.0.0/0	React Frontend Development Web Server
Custom TCP	5000	0.0.0.0/0	Node.js Express Backend REST API Endpoint
SSH (TCP)	22	0.0.0.0/0	Secure Remote Linux Terminal Management

EXPERIMENT NO. 1: AMAZON S3 CONFIGURATION & POLICIES

1. Aim: To create an Amazon S3 bucket, configure public read access policies, enable Cross-Origin Resource Sharing (CORS), and integrate the AWS SDK for seamless image upload and deletion.

2. Target S3 Resource: Bucket Name: samriddhabaket1718 | Region: ap-southeast-2

3. Step-by-Step Practical Procedure:

1. Navigated to **Amazon S3 > Buckets > Create bucket** in AWS Console.
2. Entered global unique name samriddhabaket1718 and disabled Bucket Versioning to eliminate unnecessary storage overhead.
3. Unchecked **'Block all public access'** and confirmed acknowledgment to permit direct browser retrieval of uploaded images.
4. Applied the following **JSON Bucket Policy** to grant universal s3:GetObject permissions:

```
{
   "Version": "2012-10-17",
   "Statement": [{
   "Sid": "PublicReadGetObject",
   "Effect": "Allow",
   "Principal": "*",
   "Action": "s3:GetObject",
   "Resource": "arn:aws:s3:::samriddhabaket1718/*"
   }]
  }
```

5. Configured Cross-Origin Resource Sharing (CORS) with allowed methods GET, PUT, POST, DELETE, HEAD and wildcard origin (*).
6. Implemented Node.js streaming upload using PutObjectCommand and atomic deletion with DeleteObjectCommand.

4. Result: Images stream directly into Amazon S3 with low-latency CDN performance, yielding zero server-side disk bloat.

EXPERIMENT NO. 2: POSTGRESQL & DOCKER ORCHESTRATION

1. Aim: To containerize application microservices and configure an automated, persistent PostgreSQL 16 relational database via Docker Compose.

2. Database Schema Design (PostgreSQL):

An automated table initialization routine executes on startup, provisioning two core entities:

- **Table users:** Stores user IDs, unique usernames, Bcrypt hashed passwords, user roles (user vs admin), and registration timestamps.
- **Table images:** Stores auto-incrementing IDs, image titles, original filenames, public S3 URLs, S3 object keys, uploader names, and upload timestamps.

3. Multi-Container Orchestration (docker-compose.yml):

Service Name	Base Image	Ports	Configuration & Environmental Roles
postgres	postgres:16-alpine	5432:5432	Database engine with persistent volume postgres_data and automated healthchecks.
backend	node:20-alpine	5000:5000	Express API connected to internal Docker PostgreSQL network and external AWS S3 API.
frontend	nginx:alpine (multi-stage)	80:80, 3000:80	High-performance web server hosting React production build with API reverse proxy.

4. Result: Executing sudo docker compose up -d --build builds and starts all three services in isolated, interconnected containers in under 90 seconds.

EXPERIMENT NO. 3: FULL-STACK DEVELOPMENT & RBAC

1. Frontend Implementation (React.js + Vite):

- **Drag-and-Drop Uploader:** Developed an interactive upload card with live image thumbnail preview, custom title inputs, and format validation.
- **Interactive Masonry Grid:** Displays all uploaded images with badges distinguishing S3 cloud assets from local fallback files.
- **Lightbox Modal:** Full-resolution zoom view with metadata inspection and direct links to original S3 objects.
- **Role Switcher & Status Banner:** Live chips showing real-time health connectivity for AWS EC2, Amazon S3, and PostgreSQL.

2. Role-Based Access Control (RBAC) Logic:

A core project requirement was ensuring that public users can freely upload and view images, while image deletion is restricted strictly to authorized administrators.

Role	Upload Permission	View Permission	Delete Permission	Security Mechanism
Public User	ALLOWED	ALLOWED	**DENIED** (Locked)	Read-only UI; backend returns 403 Forbidden without admin token.
Administrator	ALLOWED	ALLOWED	**ALLOWED** (Active)	Secured by Bcrypt hashed login and JWT bearer token authorization.

EXPERIMENT NO. 4: CLOUD DEPLOYMENT & LIVE VERIFICATION

1. Deployment Procedure on AWS EC2:

1. Launched an AWS EC2 instance running Ubuntu 24.04 LTS with public IPv4 address 32.236.11.79.

2. Connected via EC2 Instance Connect browser terminal and installed Docker and Docker Compose:

sudo apt update -y && sudo apt install -y docker.io docker-compose-v2 unzip

3. Downloaded the production application bundle directly from the configured S3 bucket:

curl -sO https://samriddhabaket1718.s3.ap-southeast-2.amazonaws.com/project.zip

4. Unpacked archive and deployed all microservice containers with zero downtime:

sudo docker compose up -d --build

2. Live Operational Verification:

- **Live Application Endpoint:** Verified accessible globally at **http://32.236.11.79** and port 3000.
- **Image Ingestion Verification:** Uploaded sample images from browser; binary assets successfully verified within S3 bucket samriddhabaket1718.
- **Database State Persistence:** PostgreSQL accurately logged image IDs, S3 URLs, and timestamps across container restarts.
- **RBAC Deletion Verification:** Authenticated as admin; executed deletion test. Asset was simultaneously purged from AWS S3 and deleted from PostgreSQL.

REAL-WORLD APPLICATIONS & CONCLUSION

Real-World Applications in Computer Science:

1. **E-Commerce & Digital Asset Management (DAM):** Storing, tagging, and distributing product catalogs and user-submitted review imagery.
2. **Social Media Platforms:** Scalable media ingestion pipelines with microservice-driven image transformations and CDN delivery.
3. **Healthcare & Radiology Repositories:** Secure, redundant archiving of diagnostic medical imagery adhering to data durability mandates.
4. **Enterprise Cloud Migrations:** Modernizing legacy monolithic architectures into decoupled, containerized microservices on AWS.

Internship Learning Outcomes:

• Gained practical proficiency in AWS Cloud Infrastructure (S3, EC2, IAM, Security Groups).

• Mastered containerization workflows using Docker and multi-container Docker Compose.

• Developed full-stack proficiency with React.js (Vite), Node.js (Express), and PostgreSQL 16.

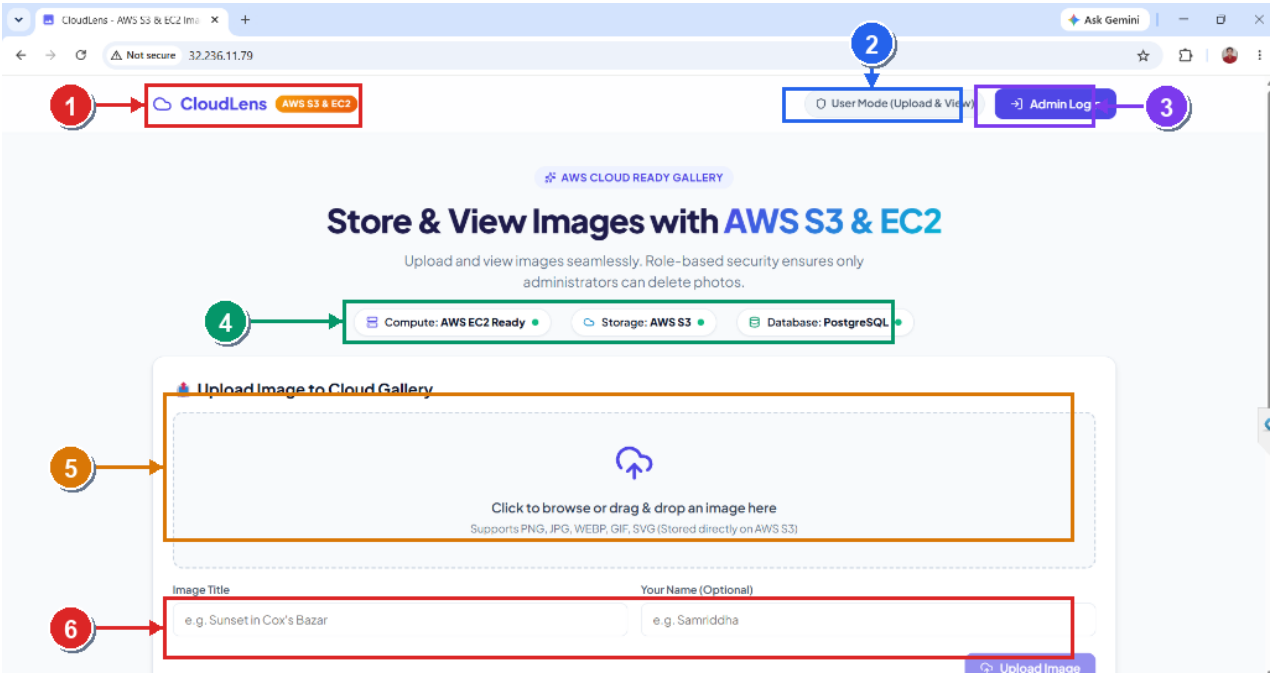
• Implemented modern software security principles including RBAC, JWT, and CORS protection.

Conclusion:

The internship on **Cloud Computing (Computer Science)** provided comprehensive, hands-on exposure to cloud architecture, containerized deployment, and full-stack software development. The resulting **CloudLens** project serves as a production-ready, highly scalable web application demonstrating best practices in cloud computing, security, and DevOps. The project is fully live, verified, and publicly accessible on AWS EC2 and version-controlled on GitHub.

EXPERIMENT NO. 5: CLOUD WEB APPLICATION INTERFACE & INGESTION TIER

The frontend client of **CloudLens** is engineered using React.js (Vite) and Tailwind CSS, presenting an intuitive, cloud-synchronized dashboard deployed live on AWS EC2 at **http://32.236.11.79**. The annotated interface below details the ingestion subsystem, health monitoring controls, and user navigation.

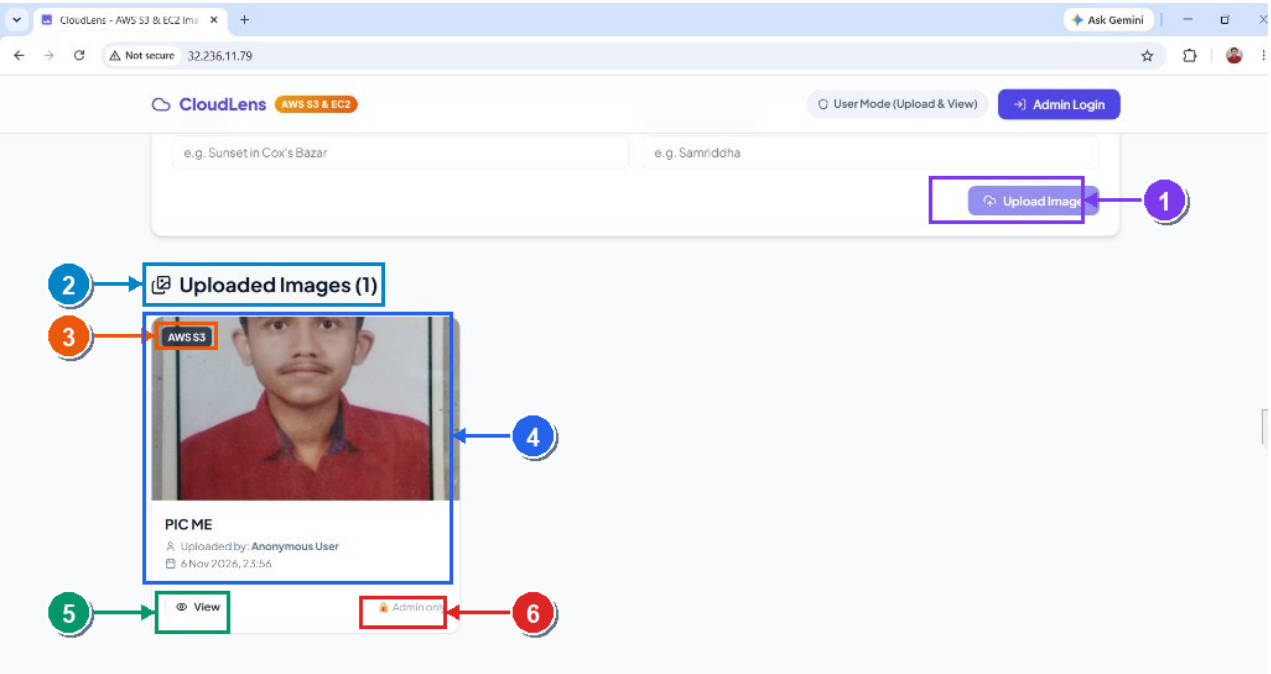


UI Component Functional Architecture & Control Reference:

Ref	UI Component / Control	Cloud Service & Operational Function
[1]	**CloudLens Brand & Architecture Badge**	Identifies application branding and verifies cloud-native microservice integration with Amazon S3 object storage and AWS EC2 compute capacity.
[2]	**Role Context Indicator (User Mode)**	Displays active authorization context (Upload & View). Guarantees least-privilege security by informing public users that delete privileges are locked.
[3]	**Admin Login Privilege Escalation Action**	Triggers modal dialog requesting JWT credentials (admin / cloudadmin123). Grants role elevation for asset management and deletion.
[4]	**Multi-Cloud Service Health Monitors**	Real-time status indicators tracking the operational connectivity of the AWS EC2 virtual machine, S3 bucket (samriddhabaket1718), and PostgreSQL container.
[5]	**Cloud Ingestion Drag & Drop Dropzone**	High-performance HTML5 drag-and-drop dropzone supporting PNG, JPG, WEBP, and GIF formats. Streams raw file buffers asynchronously to the Express API.
[6]	**Asset Metadata Parameter Inputs**	Form input controls capturing custom Image Title and optional Uploader Name. Attributes are indexed in PostgreSQL alongside the unique S3 Object Key.

EXPERIMENT NO. 6: ASSET REPOSITORY & ROLE-BASED ACCESS CONTROL (RBAC)

The gallery and asset inspection tier provides real-time cataloging of persisted images. It enforces strict Role-Based Access Control (RBAC), allowing unrestricted public inspection while isolating destructive deletion workflows behind authenticated administrative tokens.



Gallery & Security Boundary Functional Reference:

Ref	UI Component / Control	Cloud Service & Operational Function
[1]	**S3 Ingestion Execution Control (Upload Image)**	Transmits the multipart/form-data payload via asynchronous HTTP POST to /api/upload. Streams binary stream directly to Amazon S3 without intermediate disk storage.
[2]	**Persistent Cloud Asset Registry Counter**	Displays the real-time total count of indexed assets stored in the S3 bucket (samriddhabaket1718) and queried from the PostgreSQL metadata table.
[3]	**Amazon S3 Cloud Origin Indicator Badge**	Visual confirmation tag proving that the rendered media asset originates directly from Amazon S3's globally distributed object storage infrastructure.
[4]	**Cataloged Asset Thumbnail & Metadata Card**	Responsive UI card presenting the uploaded image preview, custom title (PIC ME), uploader attribution, and server-generated UTC timestamp.
[5]	**High-Resolution Lightbox Inspector (View)**	Opens an interactive full-screen modal preview displaying the uncompressed image and providing the direct, shareable AWS S3 public CDN/bucket URL.
[6]	**RBAC Deletion Lock Indicator (Admin only)**	Enforces strict zero-trust security boundaries. Unauthenticated users cannot delete images; the delete trigger is disabled until an admin JWT session is active.